Microsoft

FOUNDRY AGENT SERVICE

Capability Hosts & Private Network Injection

Best practices for sharing vs. separating Storage, Cosmos DB, and AI Search across projects

Prepared for LAM Research · Standard Agent Setup — BYO VNet + Tools behind VNet

Michael Yaacoub · **Sr Solution Engineer**

Share the platform; per-project capability hosts route each project's data

Microsoft

One network-isolated Foundry account and VNet; each project's capability host picks shared or dedicated data services.

1

Shared across the account

Foundry account, the injected VNet and agent subnet, and model deployments serve every project.

2

Isolated by default

Standard setup provisions per-project containers in your Storage and Cosmos DB — strict boundaries, no extra config.

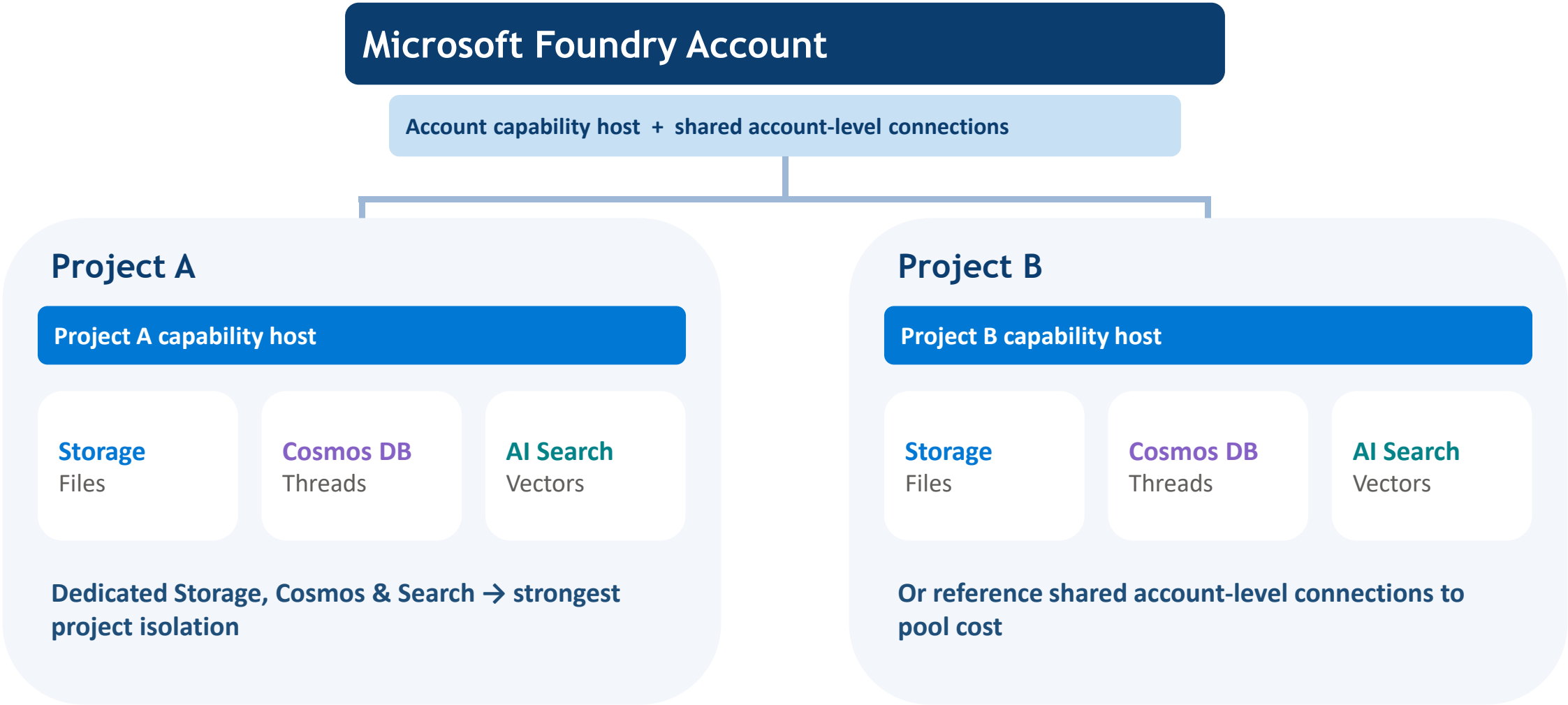
3

Chosen per resource

Each project capability host names the exact Storage, Cosmos DB, and AI Search connections it uses.

! No inheritance — a project uses only the connections referenced in its own project capability host, even if the account defines others.

Accounts contain projects; each project's capability host maps to its own data services



Two scopes bind capability hosts — but the project scope alone decides runtime data

Microsoft

Account capability host

Enables Agent Service at the account level. The request body is empty except capabilityHostKind = 'Agents'. Connections defined here are inherited by new projects — but the host configuration is not.

Project capability host

The settings Agent Service reads at runtime. Names the exact Cosmos DB (threads), AI Search (vectors) and Storage (files) connections the project uses. Nothing is auto-inherited from the account scope.

One host per scope

A second host at the same scope returns a 409 Conflict.

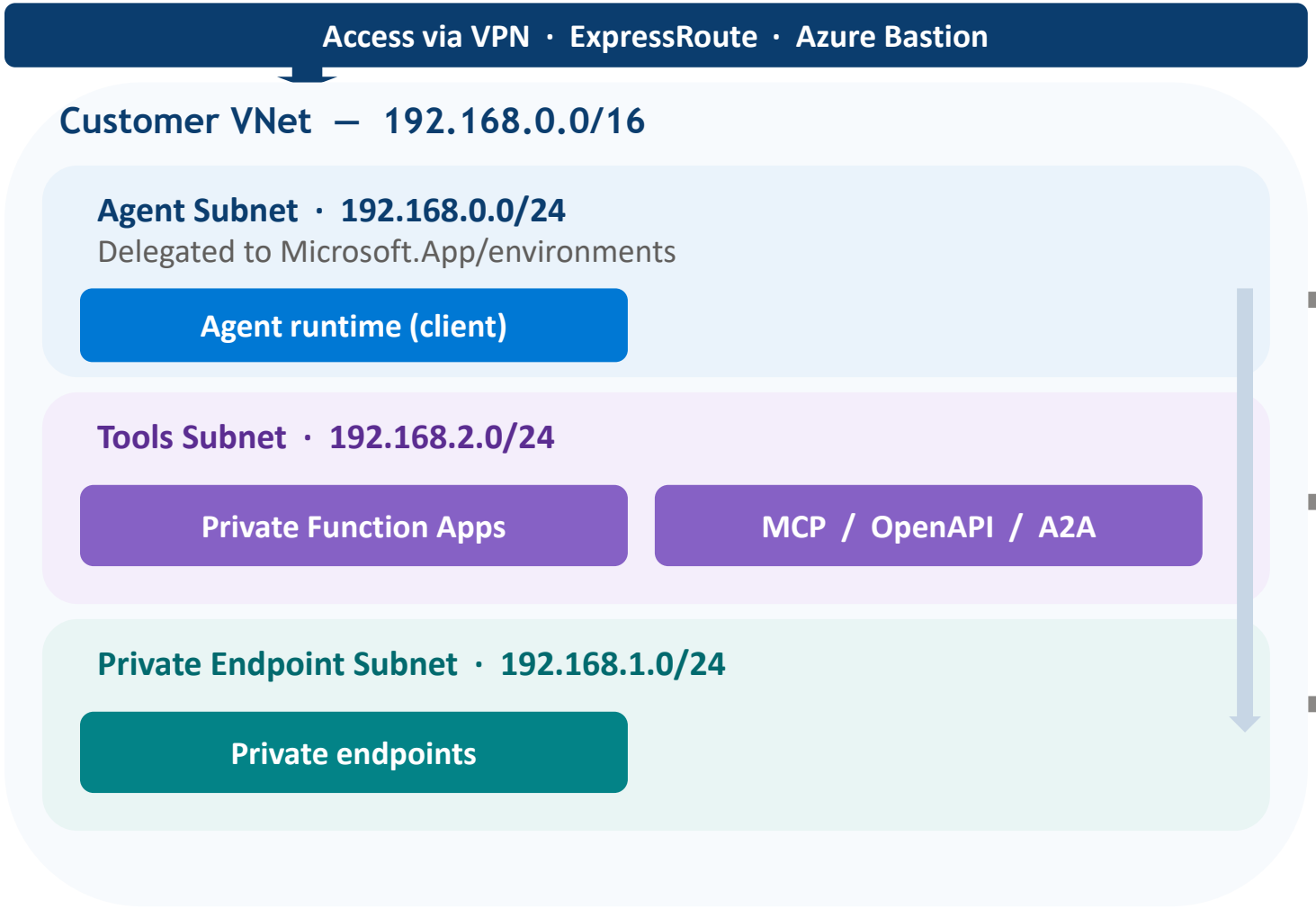
No in-place updates

Changing configuration means deleting, then recreating the host.

Account host first

A project host can't be created before an account host exists.

Network injection places agent compute in your subnet; tools and data stay private



Behind Private Endpoints
Public network access = Disabled

Azure Storage
Files & blobs · privatelink.blob

Azure Cosmos DB
Threads & agents · privatelink.documents

Azure AI Search
Vector store · privatelink.search

Agent reaches each over private DNS in the PE subnet.

No public egress — accounts and projects run with Public network access = Disabled.

Sharing pools cost and ops; separating maximizes isolation and blast-radius control

Microsoft

Service	Isolated by default?	Share across projects when…	Separate per project when…
Azure Storage	Yes — two blob containers per project	Lower ops overhead; central governance	Strict data residency; per-project blast radius
Azure Cosmos DB	Yes — three to five containers per project	Pool RU/s; fewer accounts to manage	Noisy-neighbor RU isolation; independent scaling
Azure AI Search	Partial — shared service, separate indexes	Consolidate cost; fewer services to run	Index & security isolation; per-project quotas

Default setup gives project isolation for free — choose separate resources when compliance, scaling, or blast-radius control demands it.

Each service has its own sharing playbook — shaped by how Foundry provisions it

Microsoft

Azure Storage

- Two containers per project are auto-provisioned (files + system data).
- Share the account — per-project containers already isolate data.
- Grant the MI Storage Blob Data roles; blob File Search is unsupported behind a private VNet.

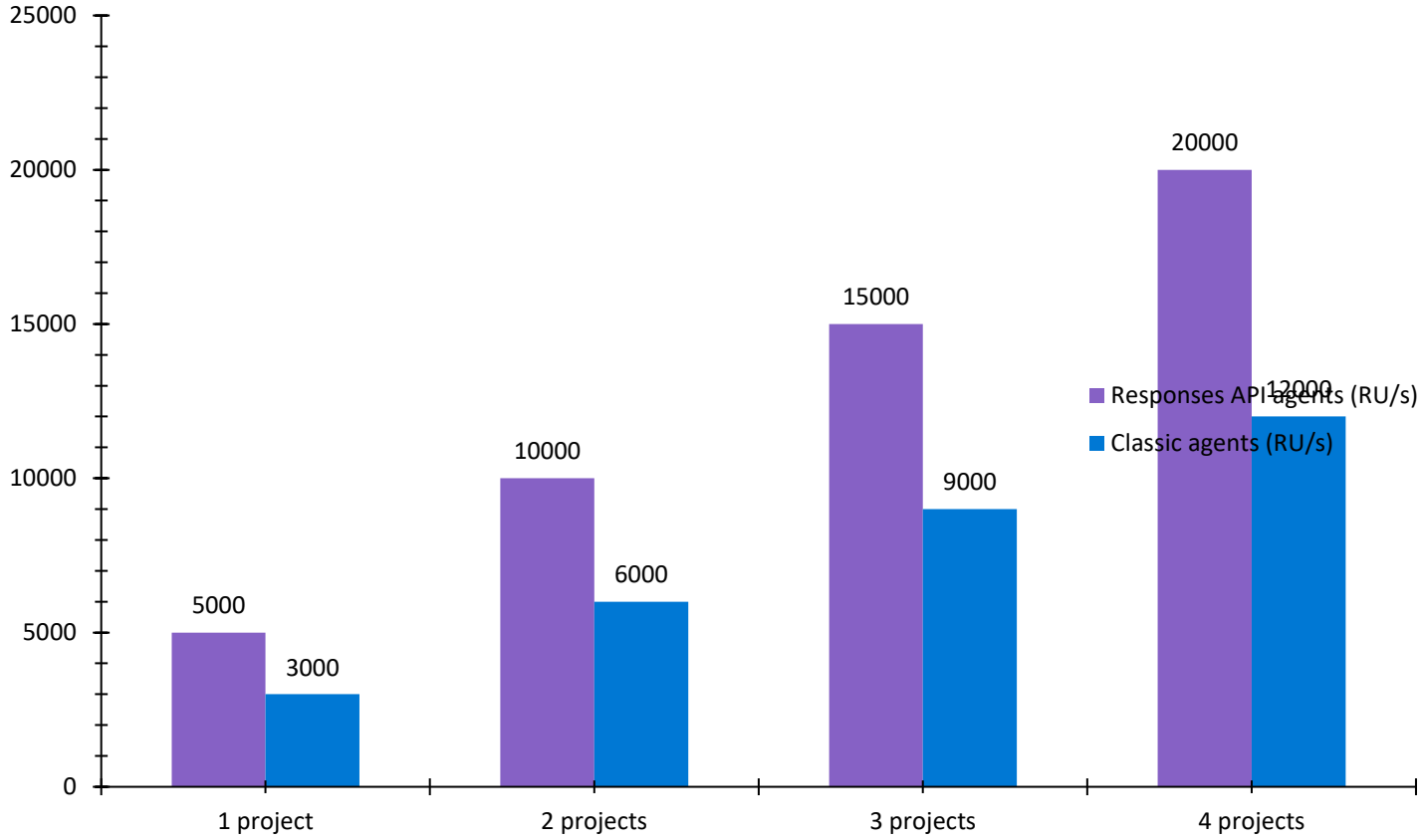
Azure Cosmos DB

- Three to five containers per project, 1000 RU/s each.
- Plan ≥ 3000 RU/s, multiplied by the number of projects when sharing an account.
- Serverless or provisioned; single-region by default — add replication manually.

Azure AI Search

- One shared service, a separate index per project.
- Grant the MI Search Index Data Contributor + Search Service Contributor.
- Use separate services only for security isolation or quota limits.

Sharing one Cosmos DB account means planning RU/s for every project it serves



Illustrative — 1,000 RU/s per container; provisioned or serverless both supported.

5,000 RU/s

per project · Responses API agents

- Minimum 3,000 RU/s total for the Cosmos DB account.
- Responses agents: 5 containers × 1000 RU/s = 5,000 RU/s per project.
- Classic agents: 3 containers = 3,000 RU/s per project.
- Too little RU/s → capability host provisioning fails.

Source: Foundry standard agent setup docs.

For LAM Research: template 19 — one isolated account, dedicated data per project

Microsoft

Use template 19 — tools behind the VNet

- Private Function Apps run in a dedicated Tools subnet (192.168.2.0/24).
- Template 15 excludes tools behind the VNet; 19 adds MCP, OpenAPI, Functions & A2A.
- Three subnets: Agent (/24, delegated), Private Endpoint, and Tools.
- BYO Storage, Cosmos DB & AI Search sit behind private endpoints.

Sharing vs. separation plan

- Share across the account: Foundry account, the VNet, and model deployments.
- Separate per project: Storage and Cosmos containers (isolated by default).
- AI Search: one service with a per-project index; split only for strict isolation.
- Size Cosmos at ~5,000 RU/s × projects; keep public access Disabled.

Net: one network-isolated account; each project's capability host points at its own data — shared only where cost and ops clearly win.

Resources — Microsoft Learn docs and GitHub deployment templates

Microsoft Learn

Capability hosts for Foundry Agent Service

learn.microsoft.com/azure/foundry/agents/concepts/capability-hosts

Set up standard agent resources

learn.microsoft.com/azure/foundry/agents/concepts/standard-agent-setup

Use your own resources

learn.microsoft.com/azure/foundry/agents/how-to/use-your-own-resources

Set up private networking (VNet injection)

learn.microsoft.com/azure/foundry/agents/how-to/virtual-networks

GitHub · microsoft-foundry/foundry-samples

Template 19 — Private network + tools behind VNet

bicep/19-private-network-agent-tools

Template 15 — Private network standard agent setup

bicep/15-private-network-standard-agent-setup

Template 17 — Private network + user-assigned identity

bicep/17-private-network-standard-user-assigned-identity-agent-setup

Terraform — BYO VNet standard agent setup

terraform/15b-private-network-standard-agent-setup-byovnet

All templates: github.com/microsoft-foundry/foundry-samples · Docs current as of July 2026.